

Local AI Stack for Pentesting Agent



This structured document outlines your project, the **Trinity Architecture**, as a high-novelty research proposal for an **IEEE conference**. It refines your focus onto **Network-Based Attacks**, which provides a concrete and measurable scope for academic defense.



Project Title: Trinity Agent

A Neuro-Symbolic Autonomous Framework for Stateful Network Penetration Testing

1. Research Overview & Novelty Factors

To qualify for an IEEE-level paper, your project must solve specific gaps in existing research. Your "Trinity" approach addresses the **Scalability-Accuracy Tradeoff** found in current monolithic AI systems.

- **Novelty 1: Neuro-Symbolic Safety Loop:** Unlike LLMs that "hallucinate" dangerous code, your agent uses a deterministic **Pydantic Guard** to enforce scope and safety rules.
- **Novelty 2: Graph-Based Knowledge Hygiene:** Instead of just listing vulnerabilities, your agent builds a **Neo4j Attack Path Map**, ensuring it only stores meaningful network relationships (e.g., *Host* → *Port* → *CVE*).

- **Novelty 3: Self-Healing Execution:** The agent autonomously detects execution errors (e.g., Nmap syntax errors) and rewrites scripts via an LLM-feedback loop, governed by a **Circuit Breaker** to prevent infinite loops.
-

2. Specialized Attack Vector: Network-Based Attacks

The agent is narrowed specifically to **Network Infrastructure Discovery and Exploitation**. This avoids the "broad-but-shallow" pitfall of general agents.

Core Network Attack Capabilities:

- **Reconnaissance (Layer 3/4):** Structured scanning of subnets to identify live hosts and open services using `python-nmap`.
 - **Vulnerability Mapping:** Cross-referencing open ports with real-time CVE feeds (NVD) stored in the **ChromaDB Vector Store**.
 - **Lateral Movement Simulation:** Using `Impacket` and `Psexec` libraries to simulate how an attacker moves between networked hosts.
 - **Protocol Exploitation:** Targeted exploitation of network protocols (SMB, RDP, SSH) via the **Metasploit RPC** interface.
-

3. Proposed Project Stages (Development Roadmap)

Stage	Focus	Key Action	Innovation Benchmark
Phase 1	Infrastructure	Deploy Neo4j, ChromaDB, and n8n in Docker.	Zero-config local data grounding.

Stage	Focus	Key Action	Innovation Benchmark
Phase 2	The Guard	Implement Pydantic AttackCommand validators.	Proving "Negative Constraints" for safety.
Phase 3	The Executor	Build the Python Subprocess runner + Error Parser.	Measure "Success-to-Retry" ratio for paper results.
Phase 4	The Brain	Orchestrate nodes in LangGraph.	Cycle-based reasoning with state persistence.
Phase 5	Evaluation	Run against a "Capture the Flag" (CTF) lab.	Compare Trinity vs. Stateless agents in path-finding.



4. Technical Integration Summary

- **Orchestrator:** LangGraph manages the state machine, allowing the agent to "backtrack" if a network path is a dead end.
 - **Intelligence:** n8n feeds the vector database daily, ensuring the agent has "fresh" knowledge of the latest network exploits.
 - **Execution:** Tools like Nmap and Metasploit are wrapped as Structured Tools, so the LLM knows exactly which flags are allowed (e.g., preventing -T5 DoS flags).
-

5. IEEE Research Paper Structure (Draft)

1. **Introduction:** Challenges of manual pentesting and the "brittleness" of current AI.
2. **Related Work:** Contrast with PentestGPT and traditional RL-based models.
3. **Proposed Architecture:** Detail the **Trinity Nodes** (Planner, Guard, Executor, Observer).
4. **Experimental Setup:** Testing the agent on a controlled network subnet (e.g., 192.168.1.0/24).
5. **Results:** Visualizing the resulting **Neo4j Attack Graph** and success rates of the Self-Healing loop.

Building an AI Cybersecurity Agent with LangGraph

This video provides a practical walk-through of the "Self-Healing" concept, showing how an agent can catch its own errors and re-generate code, which is critical for your Phase 3 development.



How to Create a
Self Healing Code
Agent - YouTube



LangChain · 23K views

